Roadmap

This page describes topics that we are currently working on and expect to advance in the next 12 months. We can't commit to specific development timelines, but community feedback on which topics are of highest interest is one factor that can help influence the prioritization of our work queue. As in any R&D project, we will react to how things evolve in many dimensions.

Open-RMF is an open-source project. We will keep developing in the open so that community members can see real-time what is happening. Let's work on it together! We encourage (and really love to see!) contributions from the community in addition to our own efforts, so if you see something that interests you, please collaborate with us through GitHub!

Scalability

When it comes to multi-robot coordination, scalability is a complex topic with many separate facets, and every one of those facets is crucial to the overall story. Concerns about scalability can be viewed in terms of **variables** and their resulting **costs**.

Variables

- **A.** Number of mobile robots sharing space
- **B.** Capabilities of each mobile robot
- **C.** Size of the space being shared
- **D.** Severity of narrow passages and bottlenecks
- **E.** System network topology (LAN, wired, Wi-Fi, remote/cloud)

Costs

- **1.** Computing resources (CPU load, RAM)
- **2.** Time to find solutions
- **3.** Quality of solutions (how close to being globally optimal, Pareto efficient, or another criteria)
- **4.** Quality of operation (responsiveness, smooth execution)

This table breaks down ongoing efforts to improve scalability, specifying which variables will scale better and which resulting costs will be improved. See below the table for detailed descriptions of each improvement.

Improvement	A	B	C	D	E	1	2	3	4
Reservation System	✅					✅	✅	✅	✅
Queuing System	✅			✅		✅	✅	✅	✅
Hierarchical Planning	✅		✅	✅		✅	✅		
Custom Negotiation		✅	✅	✅				✅	
Zenoh	✅				✅	✅	✅		✅

Reservation System

Initially described here the reservation system would assign relevant resources to robots in a globally optimal way. Originally conceived for parking spaces, it will also be applicable for elevator and workcell usage.

Queuing System

Initially described here, when multiple robots need to use one resource in a sequence (e.g. passing through a door), the queuing system will manage a rigidly defined queuing procedure instead of relying on the traffic negotiation system to resolve the use of the narrow corridor.

Hierarchical Planning

Hierarchical traffic planning would decompose the overall planning space into regions separated by chokepoints. Each chokepoint would be managed by a queue. The traffic negotiation system would only account for traffic conflicts that happen while robots transition from one queue to another queue. Reducing the scope of conflicts will make negotiations faster, less frequent, and less CPU intensive.

Custom Negotiation

Not all mobile robots or uses cases will fit neatly into the three out-of-the-box categories of "Full Control", "Traffic Light", and "Read-Only". Letting system integrators fully customize the way the fleet adapter plans and negotiates for their robot will allow Open-RMF to support more types of robots with better outcomes.

Zenoh

There are various inefficiencies in the ROS 2 implementation of Open-RMF related to DDS discovery mechanisms and the absence of message key support within ROS 2. Using Zenoh, either directly or as an RMW implementation with ROS 2, would allow for significantly more efficient communication, especially over Wi-Fi networks and for large numbers of agents and subsystems.

Tools

Managing and understanding large-scale complex systems requires tools that are able to provide comprehensive insight into how the system is operating and why. We are pushing forward efforts to develop the Site Editor into a baseline for a visualization, event simulation, and digital twin tool that can help developers and end-users alike see what is going in an Open-RMF system (whether simulated or deployed) and why. The planned capabilities are:

- Large-scale event-driven simulation (simulating hundreds of mobile robots and other Open-RMF systems in real time or faster-than-real time)
- Digital twin 3D view of a live deployment, with visualizations of decisions that are being made
- Debugging tools to look into plans and negotiations, with visualizations that explain why each solution was produced

Capabilities

To expand the scope of where and how Open-RMF can be used, we are also planning new capabilities:

- Free space traffic negotiation
- Describing tasks by drawing behavior diagrams
- Support multi-agent tasks with parallel threads of activity
- Allow constraints to be defined between tasks

Developer Experience

For wider adoption and quicker deployment, we want to improve the developer experience for system integrators.

- Simpler APIs that allow greater customization of robot behaviors
- Easier integration options, such as a "native" integration with the ROS 2 Nav Stack
  - We will provide a ROS 2 nav stack plugin that gives Open-RMF compatibility to any robot using it
- More documentation, including updating this book
- Interactive tutorials in the site editor

Found a bug? Edit this page on GitHub.

Introduction

How-to Guides provide direct and modular answers to “How-to” questions regarding specific topics you may have about Open-RMF.

They contain step-by-step instructions you can follow to quickly get started with key aspects of Open-RMF that are not covered in previous sections.

If there are any interesting topics you would like to contribute, feel free to open a Pull Request here.

Navigation Graph Strategies

This section elaborates on a few feature-driven strategies that would help optimize traffic flow. Many of them can be set up when creating your navigation graph, which is necessary for RMF integration with mobile robot fleets. The instructions and screenshots are specific to the RMF Traffic Editor tool. If this is your first time setting up a navigation graph, the Traffic Editor section in this Book goes into detail how to use the RMF Traffic Editor to create one. Do note that this is a legacy tool, and that there is an experimental RMF Site Editor under rapid development.

Important Considerations

Before setting up your navigation graph, consider the following for both physical and simulation integrations:

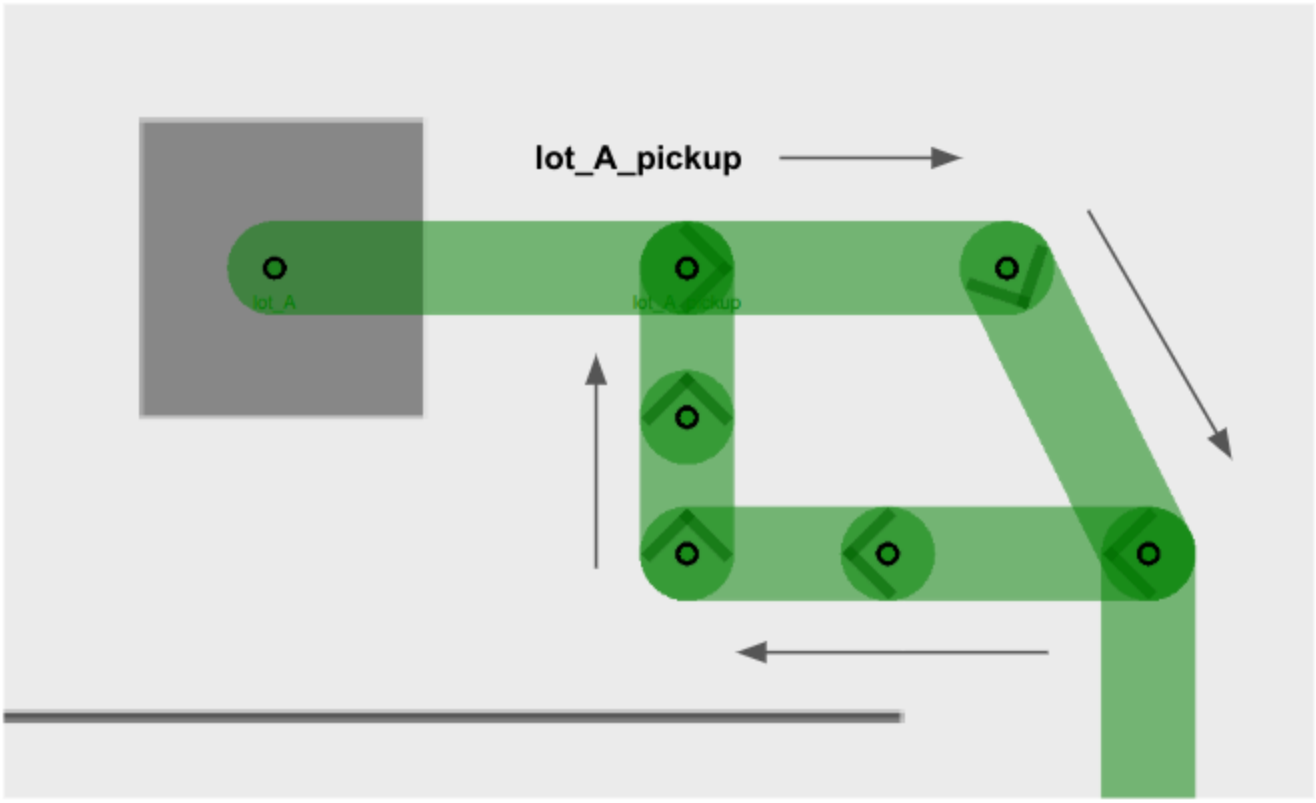
- Determine where your robots will be allowed to travel - visualize waypoints and lanes for these areas
- Plan for charger location(s) - at least one charger is required for each fleet
- Ensure sufficient space for your mobile robot to move around, leaving some room between waypoints and lanes. The amount of space required increases with the number of robots that will be integrated.
- Each area should be spacious enough to accommodate the maximum number of robots in your site. This includes adding the appropriate number of waypoints to handle these robots.
- Allocate space for holding points where possible to facilitate deconfliction

Unidirectional Lanes

Lanes in RMF can be unidirectional or bidirectional. When a lane is unidirectional, robots can only travel across the lane from a specific start waypoint to an end waypoint, essentially restricting travel to a single direction. This can come in handy for tight spaces, lift entry/exits, and other scenarios where one-way traffic is needed. For example, when multiple robots are moving within an area where space may be constrained, unidirectional lanes can be used to create a traffic flow, reducing the risk of head-on conflicts between robots.

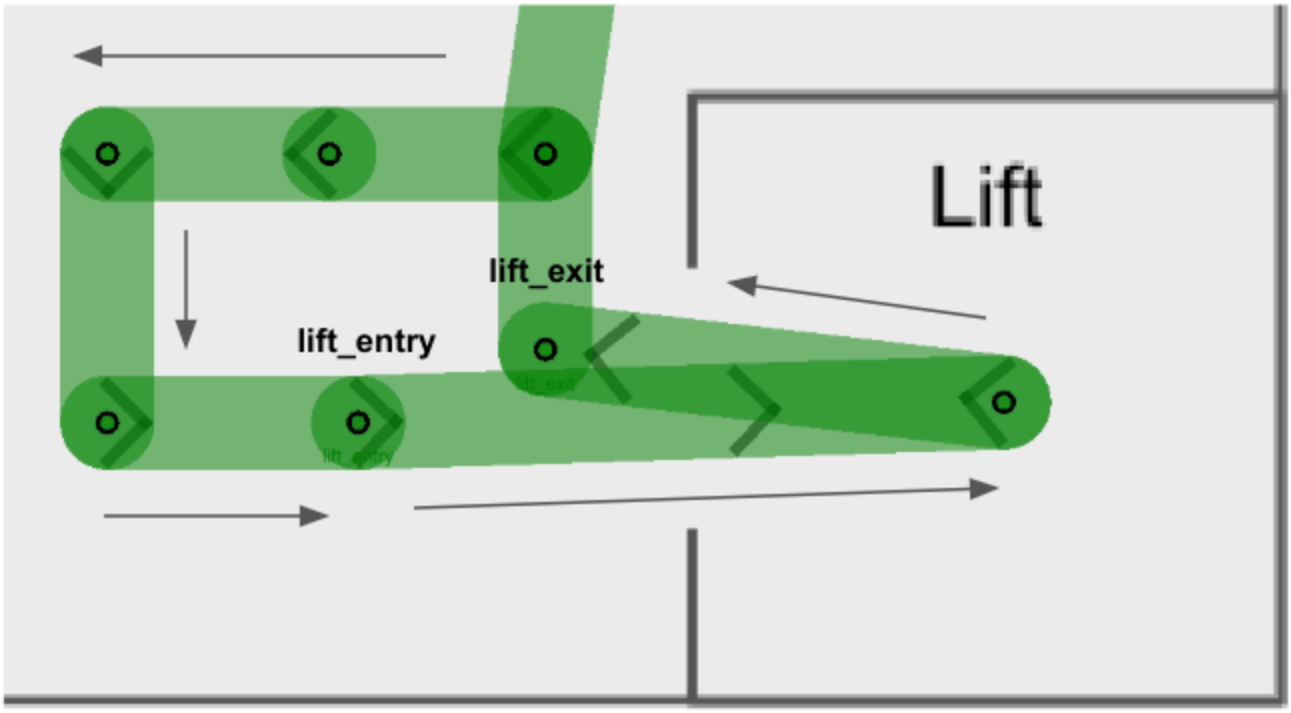
Example 1: Multiple robots visiting the same waypoint

This example illustrates multiple robots attempting to visit the same waypoint, `lot_A_pickup`, at the same time. The unidirectional lanes create a loop system that allows robots to access the waypoint one by one in a single direction.



Example 2: Lift entry and exit

Similarly, users are encouraged to make lift entry and exit lanes unidirectional, so as to facilitate robot movement in cases where a `robot_A` is exiting the lift, and a `robot_B` waiting to enter the lift. By creating separate lanes for each event and restricting their direction flow, the robots will less likely come into a deadlock and hold up an important building infrastructure that is the lift.

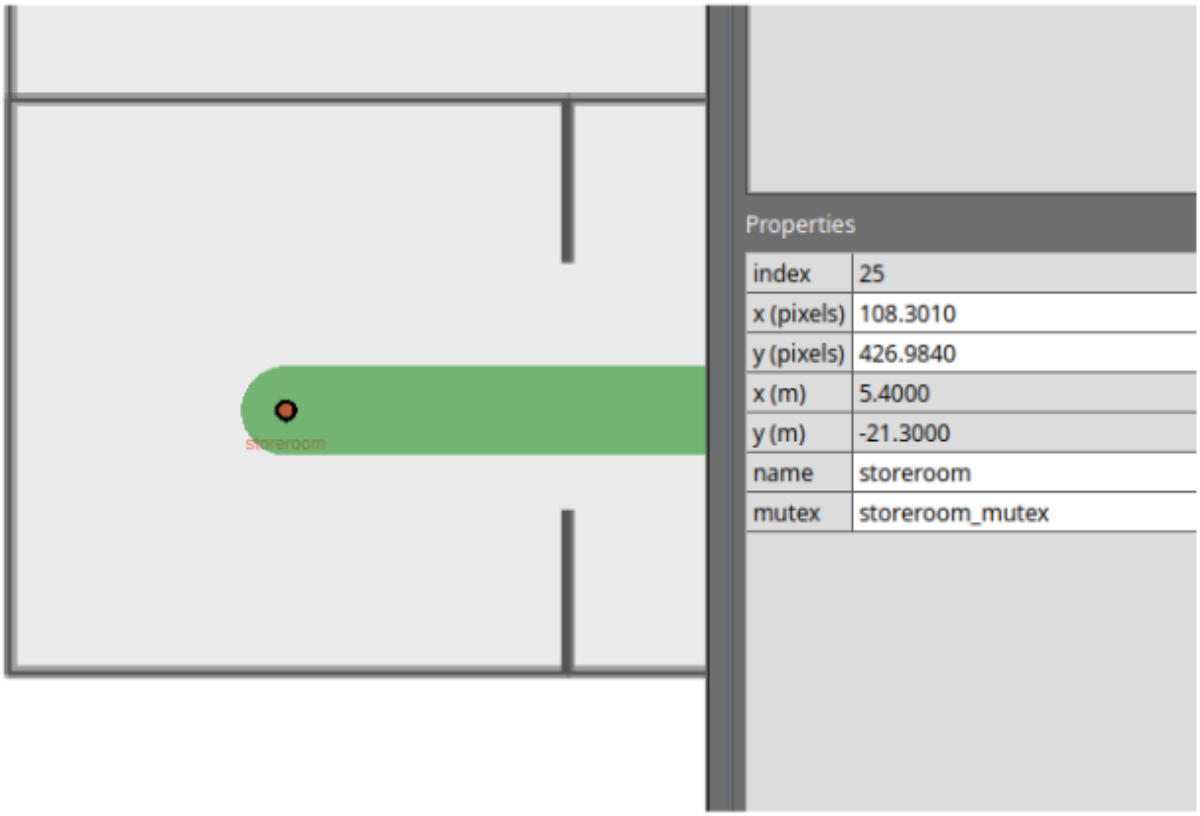


You will notice that in both examples there are several intermediate waypoints allotted. This is because these examples are designed to accommodate a site with five robots, and assigning more waypoints than the maximum number of robots help to ensure free space for each robot integrated.

Mutex Groups

The concept of mutex groups was introduced to provide users with extra control over traffic flow between robots. Waypoints and lanes can be assigned to a mutex group. **At any point of time, only one robot is allowed to occupy any waypoint or lane belonging to the same mutex group.** By implementing them stragetically in navigation graphs, users can further enhance RMF's deconfliction capabilities to prevent deadlocks and clutter in a given space.

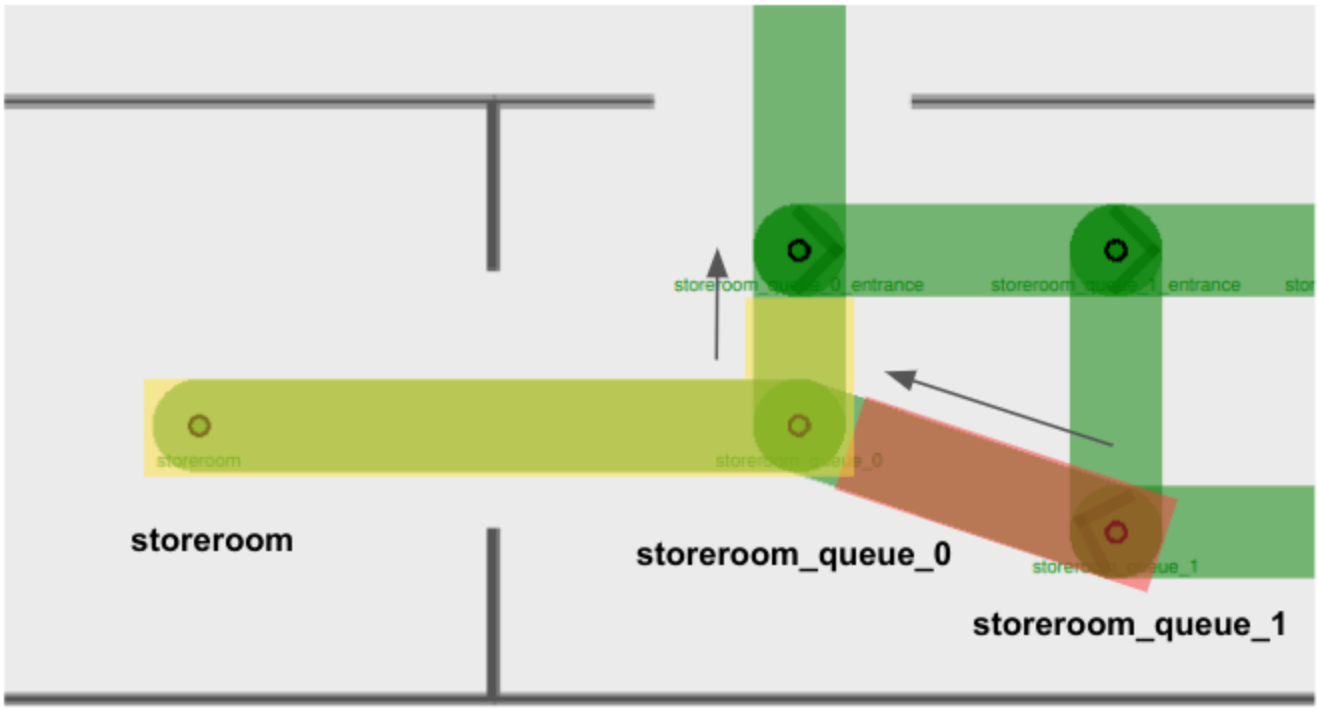
- Mutex groups can be assigned when designing your navigation graph. In the traffic editor, select an existing waypoint or lane, and add the `mutex` property to it.



- Mutex groups are differentiated by name. To assign a group of waypoints and lanes to the same mutex group, you simply have to provide a unique name when adding the `mutex` property.
- You will not be able to assign multiple mutex groups to a single waypoint or lane.
- Mutex groups are obtained on a first-come-first-served basis.

Example: Storeroom entry with queue

Imagine you have a storeroom with limited space, which only allows for one robot to enter at a time. With multiple robots tasked to enter the same room, you'd like them to queue up outside when the storeroom is occupied. You may design your navigation graph like this:



This example combines the use of unidirectional lanes (to set up a queueing system) and mutex groups (to ensure vacant waypoints for storeroom exits).

- There are two mutex groups in this example: `storeroom_mutex` highlighted in yellow, and `storeroom_queue_1_mutex` highlighted in red.
- Arrows indicate unidirectional lanes.
- Notice that both waypoints `storeroom` and `storeroom_queue_0`, as well as the lane connecting them, belong to the `storeroom_mutex` (in yellow). This ensures that when there is a `robot_A` inside the storeroom, no other robots will be allowed on the waypoint `storeroom_queue_0` despite it being unoccupied. This way, `robot_A` can smoothly exit the storeroom without facing a deadlock with an incoming robot.
- When you assign a mutex group to a waypoint, you are also encouraged to assign the same mutex group to the lane(s) it uses to leave the waypoint. This is especially important for tight spaces where you'd need to ensure that a waypoint is completely vacant before the next robot starts moving in.
  - Taking `storeroom_queue_1_mutex` (in red) as an example, if there is a `robot_B` sitting on the waypoint `storeroom_queue_1` and is moving towards `storeroom_queue_0`, the mutex in red would still be held while `robot_B` is travelling on the departure lane.
  - Other robots will not be able to begin travelling towards `storeroom_queue_1` while `robot_B` is moving away.